ADR-001 — Arquitetura, Decisões, Domínios e Operação da Solução

---

Contexto

A solução proposta tem como objetivo permitir o controle do fluxo de caixa diário de um comerciante, através do registro de lançamentos financeiros (débitos e créditos) e da geração de um consolidado diário de saldo.

O sistema deve atender aos seguintes objetivos principais:

- garantir alta disponibilidade da API de entrada
- assegurar integridade e consistência dos dados financeiros
- suportar picos de carga sem degradação significativa
- manter desacoplamento entre componentes
- fornecer rastreabilidade completa das operações
- permitir operação, monitoramento e recuperação de falhas

Além disso, a solução deve ser projetada com foco em segurança, escalabilidade e capacidade de evolução.

Decisão Arquitetural

Foi adotada uma arquitetura baseada em **Event-Driven Architecture (EDA)** com **processamento assíncrono**, utilizando uma fila de mensagens (Amazon SQS) como mecanismo de desacoplamento entre ingestão e processamento.

A arquitetura é composta por:

- API de Lançamentos (ingestão de eventos)
- Fila de Mensagens (SQS)
- Worker de Processamento (consumer)
- Banco Relacional (RDS) como fonte de verdade
- Data Mesh (S3 + Parquet) para analytics

Essa abordagem permite isolamento de responsabilidades, resiliência a falhas e escalabilidade independente.

Fundamentação das Decisões

1. Processamento Assíncrono via Fila (SQS)

Decisão

Utilizar SQS como intermediário entre a API e o processamento dos lançamentos.

Motivação

A principal motivação foi eliminar o acoplamento direto entre a camada de entrada e a persistência, garantindo:

- baixa latência de resposta da API
- absorção de picos de carga sem degradação
- resiliência a falhas do banco ou do processamento
- desacoplamento entre componentes

Além disso, a fila permite desacoplar o ritmo de ingestão do ritmo de processamento.

Trade-offs

- consistência eventual entre escrita e leitura
- necessidade de idempotência
- aumento da complexidade operacional
- necessidade de monitoramento da fila

Alternativas descartadas

Processamento síncrono (API → Banco)
Descartado por:

- alto acoplamento
- risco de indisponibilidade da API
- baixa resiliência em cenários de carga

Apache Kafka
Descartado por:

- maior complexidade operacional
- necessidade de gerenciamento de cluster
- overhead desnecessário para o cenário

---

2. API como camada de ingestão (sem persistência)

Decisão

A API não realiza persistência direta, sendo responsável apenas por validação e publicação de eventos.

Motivação

- reduzir responsabilidade da API
- evitar dependência direta de banco
- eliminar riscos de timeout
- permitir escalabilidade horizontal simples

Essa decisão garante que o serviço de entrada continue disponível mesmo em cenários de falha do banco ou do worker.

Trade-offs

- dependência do processamento assíncrono
- necessidade de controle de estado indireto

---

3. Worker como responsável pela persistência e lógica de negócio

Decisão

Centralizar no worker o processamento, validação final e persistência dos dados.

Motivação

- garantir consistência dos dados
- aplicar validações adicionais
- permitir retry controlado
- garantir que apenas eventos válidos sejam persistidos

Trade-offs

- aumento de responsabilidade do worker
- necessidade de controle de duplicidade

---

4. Uso de Banco Relacional (RDS)

Decisão

Utilizar banco relacional como fonte de verdade.

Motivação

- consistência transacional forte
- modelo adequado ao domínio financeiro

- facilidade de agregações e consultas

Trade-offs

- menor escalabilidade horizontal
- necessidade de tuning de conexões

Alternativas descartadas

DynamoDB

- modelagem complexa para agregações
- menor aderência ao modelo financeiro

---

5. Separação entre Camada Operacional e Analítica (Data Mesh)

Decisão

Utilizar S3 como Data Mesh para analytics.

Motivação

- evitar impacto de consultas analíticas no banco transacional
- permitir escalabilidade independente
- reduzir custo de consultas analíticas

Trade-offs

- duplicação de dados
- necessidade de pipeline de dados

Arquitetura e Domínios

Domínios Funcionais

Controle de Lançamentos

- ingestão de eventos
- validação inicial
- publicação na fila

Processamento e Consolidação

- consumo de eventos

- persistência
- cálculo de saldo

Analytics

- leitura de dados consolidados
- geração de relatórios

Limites de Responsabilidade

- API → ingestão e validação
- SQS → transporte confiável
- Worker → processamento e persistência
- RDS → estado consistente
- Data Mesh → dados analíticos

Levantamento de Requisitos

Requisitos Funcionais

- registrar lançamentos (crédito/débito)
- validar estrutura e dados
- enfileirar eventos
- processar eventos assíncronos
- persistir lançamentos válidos
- evitar duplicidade de eventos
- consultar lançamentos processados
- consultar saldo diário consolidado
- exportar dados para analytics
- permitir reprocessamento de eventos
- registrar origem do lançamento
- permitir integração com sistemas externos

Requisitos Não Funcionais

- alta disponibilidade da API
- desacoplamento entre serviços
- tolerância a picos (50 req/s)
- perda máxima de 5% sob falha
- consistência dos dados persistidos
- segurança em múltiplas camadas
- observabilidade completa

- escalabilidade horizontal
- resiliência a falhas
- idempotência
- isolamento entre contas AWS
- rastreabilidade completa

Priorização de Requisitos

Prioridades

1. Disponibilidade da API de entrada
2. Integridade dos dados
3. Desacoplamento
4. Escalabilidade
5. Observabilidade

Fora do escopo inicial

- consistência imediata de leitura
- auditoria completa (event sourcing full)
- versionamento avançado de eventos
- regras fiscais complexas
- múltiplas moedas

Riscos e Limitações

- atraso na disponibilidade dos dados para leitura
- dependência do worker para persistência
- necessidade de tratamento de DLQ
- maior complexidade operacional

Modelo de Evento de Lançamento

Contexto

O sistema é orientado a eventos, sendo o evento a principal unidade de comunicação.

Estrutura Completa

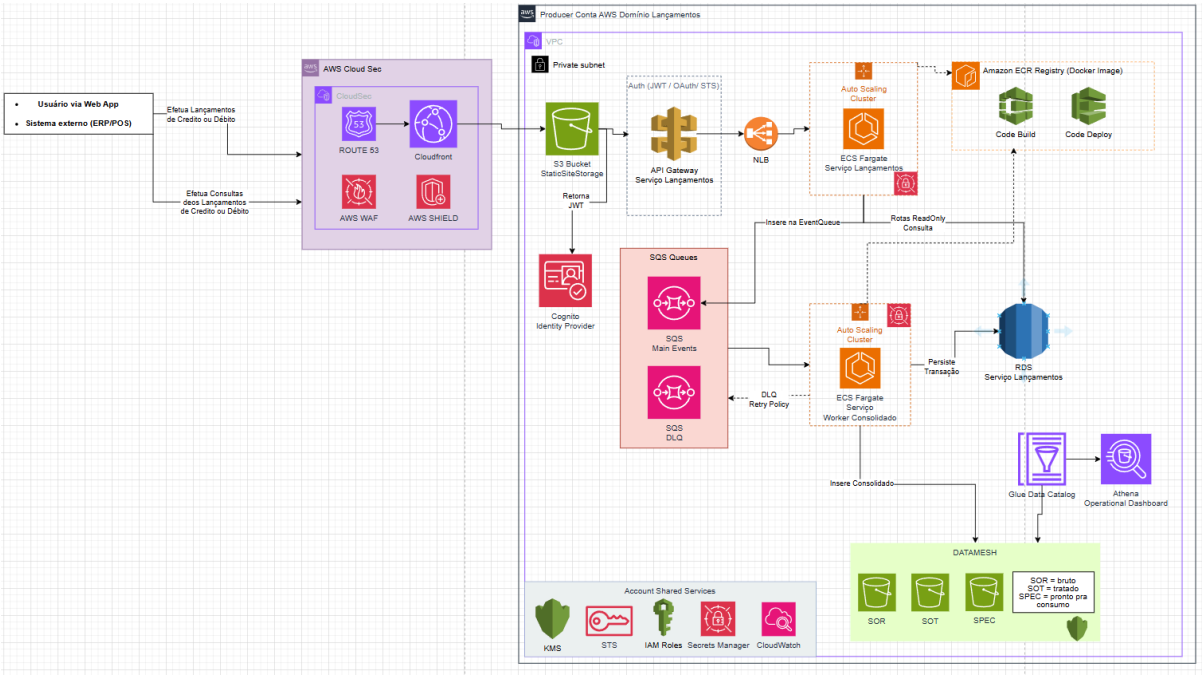

```
{
  "eventId": "uuid-v4",
  "eventType": "TRANSACTION_CREATED",
  "eventVersion": "1.0",
  "eventTimestamp": "2026-04-27T14:32:10Z",
  "source": "lancamentos-api",
  "correlationId": "uuid-v4",
  "traceId": "uuid-v4",

  "payload": {
    "transactionId": "uuid-v4",
    "externalId": "string-opcional",
    "type": "CREDIT",
    "amount": 150.75,
    "currency": "BRL",
    "description": "Venda de produto",
    "category": "RECEITA",
    "transactionDate": "2026-04-27",
    "createdAt": "2026-04-27T14:31:55Z",

    "account": {
```

```
    "accountId": "uuid-v4",
    "accountType": "CASH",
    "ownerId": "uuid-v4"
  },

  "origin": {
    "channel": "WEB",
    "ip": "192.168.0.1",
    "userAgent": "Mozilla/5.0"
  },

  "metadata": {
    "tags": ["venda"],
    "notes": "Pagamento via pix"
  }
 }
}
```

Justificativas

- `eventId` → idempotência
- `eventVersion` → evolução de schema
- `traceId` → observabilidade
- `amount positivo + type` → evita erro de cálculo
- `metadata` → extensibilidade

C3 — Arquitetura de Solução



O diagrama deve representar:

- componentes internos da API
- fluxo de publicação de eventos
- consumo pelo worker
- persistência no banco
- integração com Data Mesh

Operação da Solução

Estratégia de Deploy

A solução utiliza pipeline automatizado:

- CodeBuild → build da imagem
- ECR → armazenamento
- CodeDeploy → deploy no ECS
- CodePipeline → orquestração

Estratégia:

- rolling update
- rollback automático

Monitoramento

- métricas da API (latência, erros)
- métricas da fila (backlog, tempo em fila)
- métricas do worker
- métricas de infraestrutura

Alarmes:

- crescimento da fila
- falhas no worker
- degradação de performance

Logs

- logs estruturados em JSON
- centralização no CloudWatch
- correlação via eventId

Observabilidade

- logs + métricas + alertas
- rastreabilidade ponta a ponta
- visibilidade do fluxo de eventos

Escalabilidade

- API escala horizontalmente
- worker escala por tamanho da fila
- desacoplamento permite scaling independente

Recuperação de Falhas

- retry automático via SQS
- DLQ para mensagens falhas
- reprocessamento manual
- rollback de deploy

Referências

- Event-Driven Architecture
- CQRS
- AWS Well-Architected Framework
- Microservices Patterns

Consequências

Positivas

- alta resiliência
- escalabilidade
- desacoplamento
- operação previsível

Negativas

- maior complexidade
- necessidade de controle de consistência
- dependência de fila

Conclusão

A arquitetura proposta atende aos requisitos do desafio ao priorizar disponibilidade, integridade e escalabilidade, utilizando padrões modernos de sistemas distribuídos e boas práticas de cloud computing.